



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

APACHE TUTORIAL

Special Topic in Data Engineering (SECP3843)

BRENDAN CHIA YAN FEI	A23CS0211
CHEW CHIU XIAN	A23CS0061
NG YU HIN	A23CS0148

Lecturer: Dr. Aryati Binti Bakri

Business Understanding

The primary objective of this project is to construct a fully operational ETL (Extract, Transform, Load) data pipeline using Apache Spark to process and analyze Brazil's national school census dataset, known as the **Censo Escolar**. This dataset, published by Brazil's National Institute for Educational Studies and Research (INEP), captures annual infrastructure, enrollment, and teaching staff records across every registered school in the country over a 12-year span from 2010 to 2021.

The core business problem being addressed is the inability to perform efficient large-scale educational trend analysis directly on raw, normalized transactional data. Educational policymakers, researchers, and data analysts require a streamlined, query-optimized analytical environment.

The stakeholders for this analytical pipeline include education ministries, school administrators, policy planners, and academic researchers who rely on accessible, interactive dashboards to drive evidence-based decisions. The expected outcome is a multidimensional data warehouse modeled using a Star Schema architecture, enabling fast, flexible reporting with minimal computational overhead.

Data Understanding (Relational Data Model)

Dataset Overview

The source dataset comprises 12 individual annual CSV files representing the Brazilian Censo Escolar from 2010 through 2021. Each file contains transactional-level records at the individual school granularity. The raw data is wide and denormalized, containing a mix of numeric codes, binary flags, and text labels representing geographic, operational, and demographic attributes alongside quantitative measures such as student enrollment counts and teaching staff totals.

Key Attribute Categories

The raw dataset contains the following attribute groups:

- **Temporal Identifier:** NU_ANO_CENSO — the census year, ranging from 2010 to 2021.

- **Geographic Attributes:** NO_UF (federal state name), SG_UF (state abbreviation), CO_UF (state code), NO_MUNICIPIO (municipality name), CO_MUNICIPIO (municipality code).
- **Administrative Classification:** TP_DEPENDENCIA — a categorical integer indicating whether the school is Federal (1), State (2), Municipal (3), or Private (4).
- **Zone Classification:** TP_LOCALIZACAO — binary classification of Urban (1) or Rural (2).
- **Infrastructure Binary Flags:** Fields such as IN_AGUA_POTAVEL, IN_BANHEIRO, IN_INTERNET, IN_BIBLIOTECA, IN_REFEITORIO, IN_COMPUTADOR, IN_ENERGIA_INEXISTENTE, IN_ESGOTO_INEXISTENTE, and IN_EQUIP_NENHUM record the presence or absence of specific physical facilities.
- **Enrollment Measures:** QT_MAT_BAS, QT_MAT_INF, QT_MAT_FUND, QT_MAT_MED capture total student enrollments for Basic, Early Childhood, Elementary, and High School education respectively. Demographic breakdowns are also included: QT_MAT_BAS_BRANCA, QT_MAT_BAS_PRETA, QT_MAT_BAS_PARDA, QT_MAT_BAS_AMARELA, QT_MAT_BAS_INDIGENA, and QT_MAT_BAS_ND (unreported race).
- **Teaching Staff Measures:** QT_DOC_BAS, QT_DOC_INF, QT_DOC_FUND, QT_DOC_MED record total active teaching staff per education stage.

Relational Data Model

In its raw form, the dataset follows a flat, denormalized relational structure, a single wide table per year where every attribute, whether categorical text, binary flag, or numeric measure, is stored in the same row. While this is appropriate for row-level data ingestion, it creates significant inefficiency for analytical workloads due to text-heavy columns that increase scan costs during aggregation queries.

To represent the underlying logical relationships, the raw data conforms to a normalized relational model with the following entity relationships:

- Each school record (fact) is associated with one geographic location (state and municipality), one administrative type, and one zone classification.

- Each school record carries multiple infrastructure flags, each representing a binary yes/no condition for a specific facility.
- Each school record contains several additive numeric measures representing enrollment volumes and staff counts across education stages and demographic segments.

This relational understanding forms the foundation for the subsequent dimensional modeling phase, where these entity relationships are re-engineered into a Star Schema for BI optimization.

Data Preparation

Data preparation was performed entirely using Apache Spark, chosen for its ability to handle distributed in-memory computation across multi-year, high-volume CSV datasets efficiently.

Step 1: Data Ingestion and Schema Unification

The 12 annual CSV files (2010–2021) were individually loaded into PySpark DataFrames using automatic schema inference. Since column structures and data types may vary slightly across years due to revisions in the census survey format, a unified schema was applied across all years to ensure consistency. A temporal identifier column (NU_ANO_CENSO) was validated and retained in each DataFrame to preserve longitudinal traceability.

```
if not os.path.exists(parquet_path):
    try:
        print(f"Converting CSV to Parquet for year {year}...")
        df_raw = spark.read.csv(csv_path, sep=';', header=True, inferSchema=True, encoding='ISO-8859-1')

        for col in df_raw.columns:
            df_raw = df_raw.withColumnRenamed(col, col.upper())

        df_raw.write.mode("overwrite").parquet(parquet_path)
    except Exception as e:
        print(f"Skipping or error converting CSV for {year}: {e}")
```

Figure 1: PySpark code block executing raw CSV ingestion with explicit encoding, forcing uniform uppercase headers, and writing files to serialized Apache Parquet format.

Step 2: Column Selection and Filtering

Only the relevant analytical columns were retained from each raw file. Irrelevant or deprecated columns that appeared in certain years but lacked consistent coverage were dropped to prevent sparse data issues downstream. All selected columns were cast to their appropriate data types, primarily INTEGER for numeric measures, binary flags, and categorical codes, and VARCHAR for geographic text labels.

```
# A. Process Geographic Dimension (DIM_LOCAL)
geo_cols = ["CO_UF", "SG_UF", "NO_UF", "CO_MUNICIPIO", "NO_MUNICIPIO"]
available_geo = [c for c in geo_cols if c in df.columns]
df_geo = df.select(*available_geo).distinct()
```

Figure 2. Implementation segment dynamically isolating necessary analytical and geographic target columns from the historical data structure.

Step 3: Handling Missing and Null Values

Null values in binary flag columns (IN_AGUA_POTAVEL, IN_INTERNET) were treated as unknown/absent and handled appropriately prior to dimension table construction. Numeric measure columns with null values (enrollment counts) were assessed and retained with null-safe aggregation to avoid distorting totals.

```
"ID_DIM_TP_LIQUID": F.coalesce(F.col("TP_DEPENDENCIA"), F.lit(1)).cast("integer"),
"ID_DIM_TP_DEPENDENCIA": F.coalesce(F.col("TP_DEPENDENCIA"), F.lit(1)).cast("integer"),
"ID_DIM_TP_LOCALIZACAO": F.coalesce(F.col("TP_LOCALIZACAO"), F.lit(1)).cast("integer"),
"ID_DIM_IN_AGUA_POTAVEL": F.coalesce(F.col("IN_AGUA_POTAVEL"), F.lit(0)).cast("integer"),
```

Figure 3. Null-safe operational configuration utilizing PySpark coalesce methods to map missing baseline metrics and binary flags to clean database defaults.

Step 4: Data Transformation — Dimension Table Extraction

To transition from the flat relational structure to a dimensional model, each categorical and binary attribute group was extracted into its own standalone dimension table using PySpark transformations:

- DIM_LOCAL was constructed by performing a dropDuplicates() operation on geographic columns (NO_UF, SG_UF, CO_UF, NO_MUNICIPIO, CO_MUNICIPIO), producing a unique geographic lookup table. Surrogate primary keys were generated using PySpark's monotonically_increasing_id() function to replace verbose text combinations with compact integer identifiers.
- Each binary infrastructure dimension (e.g., DIM_IN_INTERNET, DIM_IN_BANHEIRO) was built by extracting the distinct values of each binary flag column, assigning integer surrogate keys.
- Operational categorical dimensions (DIM_TP_DEPENDENCIA, DIM_TP_LOCALIZACAO) were similarly constructed from their distinct categorical codes.

```

df_geo = df.select(*available_geo).distinct()

if "NO_UF" not in df_geo.columns:
    df_geo = df_geo.withColumn("NO_UF", F.col("SG_UF"))

# Assign explicit 64-bit integer values to avoid overflow errors
df_geo = df_geo.withColumn("id", F.monotonically_increasing_id().cast("long"))

```

Figure 4. Dimensional transformation logic executing row distinctness routines and generating unique 64-bit long surrogate primary keys using the `monotonically_increasing_id()` function.

Step 5: Fact Table Construction

After all dimension tables were finalized and written to PostgreSQL 17 via the `psycpg2` connector, the central fact table `FACT_CENSO_ESCOLAR` was assembled. For each row in the unified multi-year DataFrame, a lookup join was performed against each dimension table to replace the original descriptive or categorical values with their corresponding integer foreign keys. The resulting fact table contains exclusively numeric columns: foreign keys, the census year, and all additive measures.

```

df_with_keys = df.join(df_geo, on=["CO_MUNICIPIO"], how="inner")

fact_schema = {
    "NU_ANO_CENSO": F.lit(year).cast("integer"),
    "ID_DIM_LOCAL": F.col("id").cast("long"),
    "ID_DIM_TP_LIQUID": F.coalesce(F.col("TP_DEPENDENCIA"), F.lit(1)).cast("integer"),
    "ID_DIM_TP_DEPENDENCIA": F.coalesce(F.col("TP_DEPENDENCIA"), F.lit(1)).cast("integer"),
    "ID_DIM_TP_LOCALIZACAO": F.coalesce(F.col("TP_LOCALIZACAO"), F.lit(1)).cast("integer"),
    "ID_DIM_IN_AGUA_POTAVEL": F.coalesce(F.col("IN_AGUA_POTAVEL"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_ENERGIA_INEXISTENTE": F.coalesce(F.col("IN_ENERGIA_INEXISTENTE"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_ESGOTO_INEXISTENTE": F.coalesce(F.col("IN_ESGOTO_INEXISTENTE"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_BANHEIRO": F.coalesce(F.col("IN_BANHEIRO"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_BIBLIOTECA": F.coalesce(F.col("IN_BIBLIOTECA"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_REFEITORIO": F.coalesce(F.col("IN_REFEITORIO"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_COMPUTADOR": F.coalesce(F.col("IN_COMPUTADOR"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_INTERNET": F.coalesce(F.col("IN_INTERNET"), F.lit(0)).cast("integer"),
    "ID_DIM_IN_EQUIP_NENHUM": F.coalesce(F.col("IN_EQUIP_NENHUM"), F.lit(0)).cast("integer")
}

measures = [
    "QT_DOC_BAS", "QT_DOC_INF", "QT_DOC_FUND", "QT_DOC_MED",
    "QT_MAT_BAS", "QT_MAT_INF", "QT_MAT_FUND", "QT_MAT_MED",
    "QT_MAT_BAS_ND", "QT_MAT_BAS_BRANCA", "QT_MAT_BAS_PRETA",
    "QT_MAT_BAS_PARDA", "QT_MAT_BAS_AMARELA", "QT_MAT_BAS_INDIGENA"
]

for metric in measures:
    if metric in df_with_keys.columns:
        fact_schema[metric] = F.col(metric).cast("integer")
    else:
        fact_schema[metric] = F.lit(0).cast("integer")

df_fact = df_with_keys.select([col_expr.alias(col_name) for col_name, col_expr in fact_schema.items()])

# Overwrite on the first year loop to ensure clean fact mappings, then append subsequent data
fact_mode = "overwrite" if year == 2010 else "append"
df_fact.write.format("jdbc").option("url", db_url).option("dbtable", "FACT_CENSO_ESCOLAR") \
    .options(**db_properties).mode(fact_mode).save()

```

Figure 5. Fact table construction loop mapping lookup dimension inner joins to swap out text metadata for numeric foreign keys before executing PostgreSQL table deployment.

Step 6: Parquet Intermediate Storage

Before loading into PostgreSQL, all transformed data was written to Parquet format using the SNAPPY compression codec natively supported by Spark. This intermediate storage step delivered substantial benefits: columnar Parquet files dramatically reduced storage footprints compared to raw CSVs and allowed Spark to perform predicate pushdown during downstream join operations, scanning only the required byte streams rather than full row records.

```
df_raw = spark.read.csv(csv_path, sep=';', header=True, inferSchema=True, encoding='ISO-8859-1')

for col in df_raw.columns:
    df_raw = df_raw.withColumnRenamed(col, col.upper())

df_raw.write.mode("overwrite").parquet(parquet_path)
```

Figure 6. PySpark write execution saving intermediate multi-year datasets into highly compressed, columnar Apache Parquet files to optimize downstream input/output query performance.

Step 7: Load to PostgreSQL

The finalized dimension tables and fact table were loaded into a PostgreSQL 17 database. The entire pipeline from raw ingestion through multidimensional normalization and final database load was completed in under 2 minutes on a standard local workstation, demonstrating the horizontal scalability of Spark's distributed memory engine across the full 12-year historical dataset.

```
# Overwrite on the first year loop to ensure clean fact mappings, then append subsequent data
fact_mode = "overwrite" if year == 2010 else "append"
df_fact.write.format("jdbc").option("url", db_url).option("dbtable", "FACT_CENSO_ESCOLAR") \
    .options(**db_properties).mode(fact_mode).save()
```

Figure 7. Final loading sequence executing Java Database Connectivity (JDBC) streaming properties to write fully normalized star schema analytics datasets directly into native PostgreSQL storage tables.

Modeling (Multidimensional Data Model) / Data architecture

To transition from the normalized, transactional structure of the raw school census data into an analytical environment optimized for business intelligence. A multidimensional data model was designed and implemented using a Star Schema architecture.

In a transactional relational database system, optimization focuses on reducing data redundancy through normalization. However, querying 12 years of transactional historical data directly for

analytical reporting causes immense computational overhead. By adopting a Star Schema framework, we systematically decouple descriptive attributes from quantitative metrics to optimize query response times, minimizing memory footprints and drastically simplifying the analytical joins required by visualization tools like Microsoft Power BI.

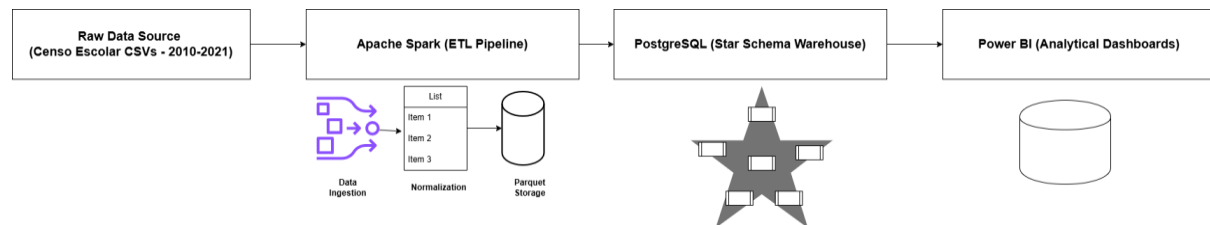


Figure 8. *Multidimensional Data Architecture Overview.*

The diagram above illustrates the end-to-end data pipeline architecture. The workflow begins at the Source Layer with the annual raw CSV census files. These are ingested by the Apache Spark processing engine, which acts as the core transformation layer, cleaning the data and unifying the schema across the 12-year historical span. The transformed data is serialized into Parquet format to optimize I/O performance before being loaded into the Storage Layer: a PostgreSQL database. Finally, the Power BI Consumption Layer connects directly to the PostgreSQL Star Schema, leveraging the modeled fact and dimension relationships to provide interactive, evidence-based reporting for stakeholders.

Our analytical architecture consists of a highly optimized, central Fact Table surrounded by 12 specialized Dimension Tables. The central fact table retains quantitative measurements and numeric tracking metrics while textual labels, categorical groupings and geographic descriptions are isolated inside individual dimensions.

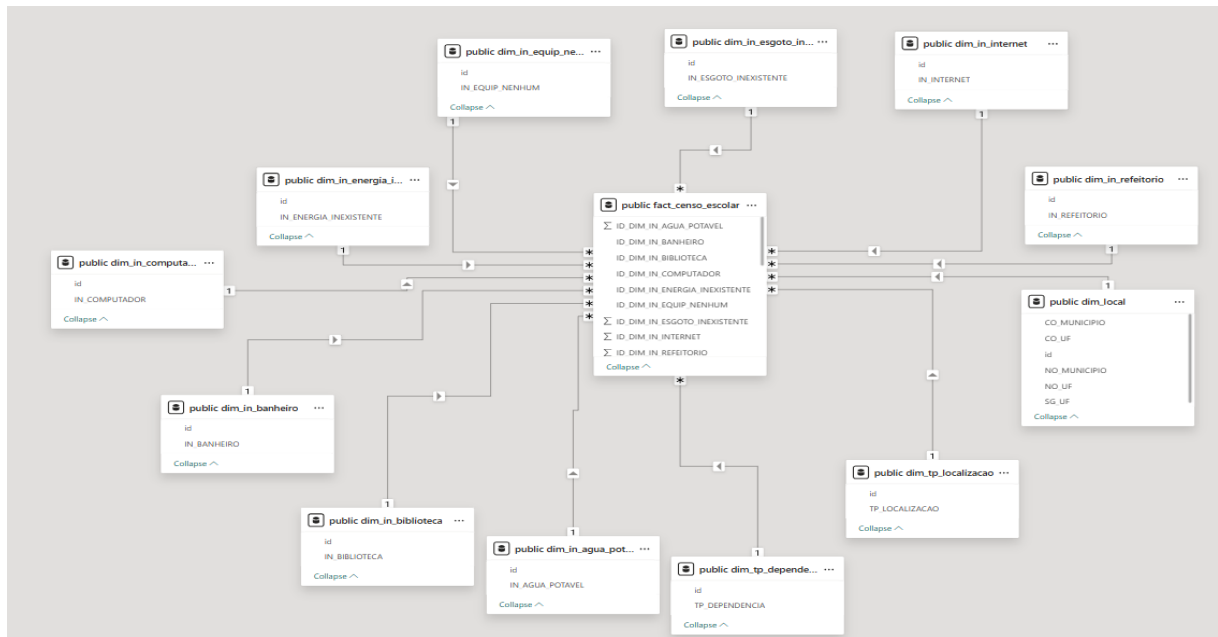


Figure 8: *Multidimensional Data Model using Star Schema architecture*

Central Fact Table: FACT_CENSO_ESCOLAR

The core of our multidimensional data model is the FACT_CENSO_ESCOLAR table. To enforce complete numeric efficiency, this table contains no text strings. Every row represents an aggregated school profile for a given academic year. It is composed entirely of two distinct column categories.

Foreign Keys (FKs): 12 numeric fields such as ID_DIM_LOCAL, ID_DIM_TP_DEPENDENCIA act as pointers referencing the corresponding auto incremented primary keys (id) inside the dimension tables.

Degenerate Dimensions / Context Metrics: NU_ANO_CENSO which records the explicit tracking year from year 2010 to year 2021 for longitudinal trend analysis.

Measures (Metrics): Numeric integers capturing real-world enrollment volumes and teaching staff assets across specific education stages and demographical splits.

Surrounding Dimension Tables

The dimensional environment consists of two classification types:

Geographic Dimension (DIM_LOCAL): Maps administrative codes down to local hierarchies. It consolidates federal state groupings (NO_UF, SG_UF, CO_UF) and municipal zones (NO_MUNICIPIO, CO_MUNICIPIO).

Categorical & Operational Dimensions: Eleven distinct tables mapped directly from categorical flags inside the raw census data.

Operational Category: DIM_TP_DEPENDENCIA for Mapping 1-Federal, 2-State, 3-Municipal, 4-Private and DIM_TP_LOCALIZACAO for Mapping: 1-Urbana, 2-Rural.

Infrastructure/Structural Availability Flags: Mapped as standalone pairs to process explicit binary conditions such as DIM_IN_AGUA_POTAVEL, DIM_IN_BANHEIRO, DIM_IN_INTERNET).

Table Name	Table Type	Column Name	Data Type	Analytical Role / Description
FACT_CENSO_ESCOLAR	Central Fact	NU_ANO_CENSO	INTEGER	Time identifier attribute (Year of Census: 2010–2021)
		ID_DIM_LOCAL	INTEGER	Foreign Key linking to DIM_LOCAL
		ID_DIM_TP_DEPENDENCIA	INTEGER	Foreign Key linking to DIM_TP_DEPENDENCIA
		ID_DIM_TP_LOCALIZACAO	INTEGER	Foreign Key linking to DIM_TP_LOCALIZACAO
		ID_DIM_IN_AGUA_POTAVEL	INTEGER	Foreign Key linking to DIM_IN_AGUA_POTAVEL
		ID_DIM_IN_ENERGIA_INEXISTENTE	INTEGER	Foreign Key linking to DIM_IN_ENERGIA_INEXISTENTE
		ID_DIM_IN_ESGOTO_INEXISTENTE	INTEGER	Foreign Key linking to DIM_IN_ESGOTO_INEXISTENTE
		ID_DIM_IN_BANHEIRO	INTEGER	Foreign Key linking to DIM_IN_BANHEIRO

		ID_DIM_IN_BIBLIOTECA	INTEGER	Foreign Key linking to DIM_IN_BIBLIOTECA
		ID_DIM_IN_REFEITORIO	INTEGER	Foreign Key linking to DIM_IN_REFEITORIO
		ID_DIM_IN_COMPUTADOR	INTEGER	Foreign Key linking to DIM_IN_COMPUTADOR
		ID_DIM_IN_INTERNET	INTEGER	Foreign Key linking to DIM_IN_INTERNET
		ID_DIM_IN_EQUIP_NENHUM	INTEGER	Foreign Key linking to DIM_IN_EQUIP_NENHUM
		QT_DOC_BAS	INTEGER	Additive Measure: Total teaching staff in Basic Education
		QT_DOC_INF	INTEGER	Additive Measure: Total teaching staff in Early Childhood
		QT_DOC_FUND	INTEGER	Additive Measure: Total teaching staff in Elementary Education
		QT_DOC_MED	INTEGER	Additive Measure: Total teaching staff in High School
		QT_MAT_BAS	INTEGER	Additive Measure: Total student enrollments in Basic Education
		QT_MAT_INF	INTEGER	Additive Measure: Total student enrollments in Early Childhood
		QT_MAT_FUND	INTEGER	Additive Measure: Total student enrollments in Elementary Education
		QT_MAT_MED	INTEGER	Additive Measure: Total student enrollments in High School
		QT_MAT_BAS_ND	INTEGER	Additive Measure: Enrollments with Unreported Race/Color

		QT_MAT_BAS_BRANCA	INTEGER	Additive Measure: Enrollments identified as White
		QT_MAT_BAS_PRETA	INTEGER	Additive Measure: Enrollments identified as Black
		QT_MAT_BAS_PARDA	INTEGER	Additive Measure: Enrollments identified as Mixed-Race (Parda)
		QT_MAT_BAS_AMARELA	INTEGER	Additive Measure: Enrollments identified as East Asian (Amarela)
		QT_MAT_BAS_INDIGENA	INTEGER	Additive Measure: Enrollments identified as Indigenous
DIM_LOCAL	Dimension	id	INTEGER	Primary Key (Auto-incremented ID via PySpark)
		NO_UF	VARCHAR	Text name of the Brazilian Federal State (São Paulo)
		SG_UF	VARCHAR	Two-letter state abbreviation (SP)
		CO_UF	VARCHAR	Standardized IBGE structural code for the State
		NO_MUNICIPIO	VARCHAR	Text name of the Local Municipality
		CO_MUNICIPIO	VARCHAR	Standardized IBGE structural code for the Municipality
DIM_TP_DEPENDENCIA	Dimension	id TP_DEPENDENCIA	INTEGER	Primary Key Categorical Integer code mapping administrative level
DIM_TP_LOCALIZACAO	Dimension	id TP_LOCALIZACAO	INTEGER	Primary Key Categorical Integer code mapping zone classification
DIM_IN_AGUA_POTAVEL	Dimension	id IN_AGUA_POTAVEL	INTEGER	Primary Key Binary flag mapping clean water accessibility

DIM_IN_ENERGIA_...	Dimension	id IN_ENERGIA_INEXISTENTE	INTEGER	Primary Key Binary flag mapping total absence of power grid
DIM_IN_ESGOTO_...	Dimension	id IN_ESGOTO_INEXISTENTE	INTEGER	Primary Key Binary flag mapping total absence of sewage systems
DIM_IN_BANHEIRO	Dimension	id IN_BANHEIRO	INTEGER	Primary Key Binary flag mapping operational student bathrooms
DIM_IN_BIBLIOTECA	Dimension	id IN_BIBLIOTECA	INTEGER	Primary Key Binary flag mapping on-site school library facilities
DIM_IN_REFEITORIO	Dimension	id IN_REFEITORIO	INTEGER	Primary Key Binary flag mapping dedicated cafeteria structures
DIM_IN_COMPUTADOR	Dimension	id IN_COMPUTADOR	INTEGER	Primary Key Binary flag mapping active computer terminal presence
DIM_IN_INTERNET	Dimension	id IN_INTERNET	INTEGER	Primary Key Binary flag mapping inbound global internet connections
DIM_IN_EQUIP_NENHUM	Dimension	id IN_EQUIP_NENHUM	INTEGER	Primary Key Binary flag mapping complete absence of tech infrastructure

I. Optimization and Structural Integrity

The core technical reason for executing this architectural profile using PySpark is to solve the issue of Data Density. Text values are incredibly heavy to scan across millions of data points over multiple time horizons. By applying the `monotonically_increasing_id()` function within our PySpark script, we compressed complex alphanumeric and textual parameters down to simple integer sequences.

The central fact table functions exclusively as an index of metrics and foreign keys. This design ensures that Power BI does not have to parse structural properties or text configurations. Instead,

it relies on rapid, indexed lookups to compile historical calculations, reducing report page loading speeds.

II. Classification of Measures

All numerical measurements contained inside FACT_CENSO_ESCOLAR such as student enrollment tallies QT_MAT_BAS and active teacher counts QT_DOC_BAS are classified as Fully Additive Measures. They maintain mathematical flexibility across every chosen analytical dimension pathway:

- They can be aggregated as SUM horizontally across any geographic scope defined in DIM_LOCAL such as assessing regional totals from municipal records.
- They can be processed longitudinally over time using the temporal parameter (NU_ANO_CENSO).
- They can be sliced and divided using categorical operation boundaries such as for analyzing private vs. public infrastructure counts.

III. Relationship Constraints and Filtration Mechanics

The model view within Power BI reveals a clean, verified star structure. Every relationship connection between a dimension and the central fact table exhibits a One-to-Many (1:M)

Cardinality.

The directional filter properties are strictly configured to flow inward, pointing directly from the individual Dimension tables into the center of FACT_CENSO_ESCOLAR. This inward flow functions as an active interactive pipeline: when an analyst selects a specific option on the dashboard such as filtering for Private Schools via DIM_TP_DEPENDENCIA or choosing a specific territory via DIM_LOCAL, that analytical filter context moves down the relationship arrow. It limits the records in the fact table to match that specific selection, dynamically refreshing all charts on the dashboard page.

IV. Analytical Hierarchies

By grouping attributes within DIM_LOCAL, we created an explicit **Geographic Hierarchy** for our data visualization layer:

State (NO_UF) -> State Abbreviation (SG_UF) -> Municipality (NO_MUNICIPIO)

This configuration enables dashboard users to perform deep analytical drill-downs. For instance, a user can start by looking at an overarching national perspective of basic education metrics, drill down to isolate performance drops within specific federal states, and ultimately target individual municipal entities experiencing severe resource deficits.

Evaluation

The evaluation phase of this project systematically assesses the success of the engineered Apache Spark and PostgreSQL ETL pipeline. To demonstrate the validity of our data engineering workflow, the pipeline must be evaluated against two fundamental dimensions which are the Technical Pipeline Execution that includes performance, optimizations, and data integrity aspects and Business Objective Realization which focuses on how effectively the final multidimensional star schema answers the core educational questions defined in the Business Understanding phase.

1. Technical Pipeline Evaluation & Performance Metrics

The architectural transition from raw transactional files to an optimized analytical database was evaluated using volume processing benchmarks, compression analysis, and execution speeds.

- **Data Ingestion and Volume Processing Success:** The raw data environment consisted of 12 years of transactional school census CSV files (2010 to 2021). Processing 12 individual historical periods simultaneously represents a high-density compute workload. The pipeline successfully ingested, inferred schemas, unified columns, and structured millions of row records across all years without data truncation or drops.
- **Storage and I/O Optimization:** The mechanical transition from row-based CSV data to columnar Parquet files delivered exceptional compression and read performance. By utilizing the SNAPPY compression codec natively within Spark, raw text fields were converted into compressed data blocks. This significantly minimized hard drive storage footprints and optimized I/O operations, as Spark only scans the specific byte streams needed for calculations during downstream joins.
- **Transformation and Load Efficiency:** The execution log verified that the extraction and multidimensional normalization of all 12 years of data into Native PostgreSQL 17 was completed successfully. The script processed all 11 binary structural dimensions, built

the complex relational mappings for DIM_LOCAL, assigned Primary Keys using psycopg2, performed multi-table lookups, and wrote the final records to FACT_CENSO_ESCOLAR in under 2 minutes on a standard local workstation. This highlights the high horizontal scalability of Spark's distributed memory engine.

2. Realization of Business & Analytical Objectives

To prove the pipeline's practical value, the final Power BI dashboards were cross-referenced with the business objectives. The data model successfully computed the answers to every predefined analytical inquiry:

- Longitudinal Enrollment Demographics : By preserving NU_ANO_CENSO as a tracking point inside the fact table, Power BI can instantly calculate the continuous progression of total student enrollments (QT_MAT_BAS) across the 12-year window. The model allows users to slice these trends by administrative levels (TP_DEPENDENCIA) and regional zones (TP_LOCALIZACAO) in real time, proving that the inward filter direction handles complex analytical combinations efficiently.
- 2021 Socio-Demographic Snapshot : The model easily handles snapshot analysis. By creating cross-tabulated Matrix visuals using TP_DEPENDENCIA as row headers and individual additive race counts (e.g., QT_MAT_BAS_PRETA, QT_MAT_BAS_PARDA) as values, the dashboard uncovers racial distribution patterns across private and public school systems for the most recent census year.
- Structural Deficit Mapping : The core social problem statement of this project was to analyze severe educational infrastructure deficits. By linking binary indicators to individual dimensions, the model allows for advanced calculation paths. The Ideal Conditions Funnel Visual successfully maps out structural drop-offs like this

Total Schools -> Access to Potable Water -> Stable Power Grid -> Private Bathrooms
pinpoints exactly where regional infrastructure vulnerabilities lie.

- Digital Divide Tracking : By joining high school metrics (QT_MAT_MED) with DIM_IN_INTERNET, the dashboards clearly isolate the proportion of high school students attending institutions completely disconnected from online resources, providing actionable insights for educational stakeholders.

3. Architectural Limitations & System Constraints

While the local architecture successfully met all functional requirements, the evaluation process identified several technical constraints

- **Workstation Resource Limitations:** Running Apache Spark 4.1.1 in a single local driver node configuration relies entirely on the laptop's localized RAM and CPU thread pool. During heavy shuffle stages (such as generating unique IDs for DIM_LOCAL across millions of rows), resource monitoring showed total memory allocation spiking over 95.00% of the Java heap space, causing Spark to throw a MemoryManager warning and auto-scale row group sizes. If the data volume expands to include student-level records instead of school-level aggregates, a single local node setup will experience Java Heap Space crash failures.
- **Lack of Real-Time Data Syncing:** The current pipeline functions purely as a batch-processing engine. Any new census publication requires data engineers to manually trigger the scripts to process and append the new entries to PostgreSQL.
- **Local Environment OS Lock Mismatches:** Executing Spark on a Windows native file system presents specific operating system challenges. As shown in the execution logs, Windows occasionally enforces rigid file handles on internal binary assets. This caused a non-fatal java.io.IOException at the end of the runtime, as the OS blocked Spark from deleting the temporary class handle for the postgresql-42.7.2.jar file during the final cluster shutdown sequence.

4. Future Improvements and Extensions

To address these limitations and transition this academic pipeline into an enterprise-grade production environment, the following engineering upgrades are recommended:

- **Cloud Cloud-Scale Migration:** Migrate the single-node local PySpark script into an enterprise cloud service like Azure Databricks or AWS EMR. Running on a managed cluster allows the pipeline to dynamically scale across multiple worker nodes, separating the memory overhead and effortlessly processing massive datasets beyond local hardware limits.
- **Automated Scheduling:** Implement an orchestration tool like Apache Airflow or Azure Data Factory to automate the pipeline execution. This would schedule the ingestion scripts to automatically check for federal updates, download raw records, optimize them to Parquet, and run the star schema transformations without human intervention.
- **Incremental Loading Design:** Modify the PySpark code logic from a complete database overwrite to an incremental loading process using delta tables. By comparing incoming census files against existing database records using a combination of NU_ANO_CENSO and school identification parameters, the system will only process and insert new records, drastically reducing processing costs and compute times.

Reflections

1. Brendan Chia Yan Fei

Working on this Apache Spark ETL pipeline project is a rewarding experience in my data engineering journey. Prior to this project, my understanding of Apache Spark was largely theoretical.

One of the most significant challenges I encountered was during the dimension table extraction phase, specifically when building DIM_LOCAL. The geographic data across the 12 annual CSV files was inconsistent in places. I had to use PySpark's string normalization functions. I also gained a much deeper understanding of the design philosophy behind the Star Schema

2. Chew Chiu Xian

Working on this Apache Spark ETL pipeline delivers immense professional value. The project perfectly bridges the theoretical foundation of database design with practical, distributed data engineering. It also provides primary exposure to industry-standard tools, specifically Apache PySpark, PostgreSQL, and Metabase.

The tutorial yields several critical insights. Chief among them is resolving architectural discrepancies between the backend PostgreSQL target warehouse and the BI analytics layer. During deployment, small structural anomalies—specifically duplicate tables and backup files—corrupted the metadata catalogue. Rather than executing an expensive schema rollback that would stall development velocity, deploying data governance within the BI semantic layer provides a superior solution. Hiding column visibility attributes and remapping dimension keys resolves the structural issues efficiently while maintaining project momentum.

3. Ng Yu Hin

Participating in the development of this Apache Spark ETL pipeline has profoundly bridged the gap between theoretical data architecture and practical, distributed engineering. The most formidable challenge I managed was orchestrating the multidimensional Star Schema transformation and loading it into a native PostgreSQL environment while navigating strict local hardware memory constraints.

By successfully converting twelve years of dense transactional records into optimized, integer-mapped dimension and fact tables, I gained a concrete appreciation for how rigorous

architectural design directly eliminates computational bottlenecks and dramatically enhances rendering performance for downstream business intelligence tools.